



UNIVERSITY OF TRENTO
MASTER'S DEGREE IN DATA SCIENCE

~ · ~

2025-2026

Secure LLM orchestration for enterprise workflows using the Model Context Protocol

Supervisor
Prof. Daniele Miorandi

Graduate Student
Ettore Miglioranza
257311

FINAL EXAMINATION DATE: July 3, 2026

Dedica

Acknowledgments¹

¹In the interest of transparency, the author discloses that portions of the written text of this thesis were drafted and edited with the assistance of a generative artificial intelligence system, used as a writing aid under the author's continuous direction, review, and final approval. The research problem, system architecture, design decisions, implementation, and experimental validation presented in this work are the author's own. The author takes full responsibility for the accuracy and content of the entire document.

Abstract

Large language models (LLMs) have demonstrated cutting-edge capabilities in understanding intent and semantic routing, but their adoption in business workflows remains limited by stringent privacy and security requirements. This thesis investigates how LLM-based orchestration can be implemented in a real-world business context, whilst providing a formal guarantee that no personally identifiable information (PII) crosses the boundaries of an external model. The work focuses on a specific workforce management domain within the Antares ERP ecosystem, where natural language requests must be reliably mapped to operational actions, such as locating nearby workers on client sites, identifying colleagues based on proximity, and validating absence cover and substitution chains.

The central research question is: how can an external LLM be integrated as a semantic router without exposing sensitive business data, whilst ensuring deterministic and verifiable execution of business tools? To answer this question, the thesis proposes a ‘privacy-by-design’ architecture based on the Model Context Protocol (MCP). The system enforces a strict separation between language understanding and data access through a reversible anonymisation pipeline: user requests are pseudonymised before reaching the model, routed by the LLM using only anonymised text, and de-anonymised exclusively on the server side at the time of tool execution. Entity detection is performed by a multilingual NER microservice based on GLiNER, chosen over rule-based alternatives such as Microsoft Presidio precisely because of its robustness in processing business inputs in Italian. Token-to-entity mappings are stored in a short-lived Redis context, ensuring that pseudonymisation is both ephemeral and fully reversible. The MCP server exposes schema-validated, typed tools to the model and enforces strict routing constraints, whilst a .NET 10 client orchestrator manages the end-to-end request lifecycle.

The approach draws on the broader state of the art in privacy-preserving LLM infrastructure. In particular, OpenAI’s Privacy Filter (Apache 2.0, 2025), recently made open source, confirms that lightweight, on-premise NER models incorporated as pre-processing layers represent the emerging industry standard for PII containment. The key architectural contribution of this thesis relative to that reference model is *reversibility*: whilst one-way encoding destroys the mapping between placeholders and entities, the token-based scheme proposed here preserves full operational correctness whilst allowing controlled de-anonymisation at the protocol boundary, a distinction relevant under Article 4.5 of the GDPR, which classifies such processing as pseudonymisation rather than anonymisation.

The thesis makes three concrete contributions: (1) a multi-service architecture that structurally isolates sensitive data from the LLM context, (2) a deterministic and reversible privacy layer based on token mapping that preserves the correctness of business logic throughout the entire request lifecycle, and (3) a validated set of realistic workforce management use cases demonstrating the effectiveness of the approach in a representative production environment. The validation confirms that the accuracy of semantic routing and NER performance on Italian-language inputs are sufficient for enterprise deployment and that no raw PII is observable within the boundaries of the tested LLM context.

This work positions MCP as a practical and applicable interface standard for enterprise AI orchestration and highlights the key engineering trade-off inherent in privacy by design: increased architectural complexity is the necessary cost for demonstrable privacy assurance, but it unlocks reliable and verifiable LLM-driven workflows without violating data protection constraints.

Keywords: LLM orchestration, Model Context Protocol (MCP), privacy-by-design, PII

pseudonymization, semantic routing, named entity recognition, GLiNER, enterprise AI security, GDPR compliance

Contents

1	Introduction	1
1.1	Context and Motivation	1
1.2	The Problem of Data Privacy in LLM Integration	1
1.3	Research Objectives and Questions	2
1.4	Contributions	2
1.5	Structure of the Thesis	3
2	Background and Related Work	5
2.1	Large Language Models in Enterprise Workflows	5
2.1.1	Semantic Routing and Intent Classification	6
2.1.2	Tool Calling and AI Agents	6
2.2	The Model Context Protocol (MCP)	7
2.2.1	Standardizing LLM Interfaces	7
2.2.2	Security Boundaries in MCP	7
2.3	Privacy-Preserving Natural Language Processing	8
2.3.1	Personally Identifiable Information (PII) Definitions	8
2.3.2	Named Entity Recognition (NER) for Data Masking	8
2.3.3	Rule-based vs. Deep Learning Approaches (Presidio vs. GLiNER)	9
2.4	State of the Art in Secure LLM Orchestration	9
2.4.1	OpenAI Privacy Filter: A Reference Open-Weight Model for PII Redaction	11
3	System Architecture and Design	13
3.1	Architectural Principles: Privacy-by-Design	13
3.2	The Antares AI Ecosystem	14
3.3	Functional and Non-Functional Requirements	14
3.4	High-Level System Architecture	15
3.5	Component Design	15
3.5.1	The MCP Client Orchestrator	15
3.5.2	The Anonymization Service API	17
3.5.3	The GLiNER-based NER Microservice	17
3.5.4	The Antares MCP Server	18
3.6	Data Flow and Lifecycle of a Request	18
3.6.1	Tokenisation and Ephemeral Context Storage	18
3.6.2	Isolated Semantic Routing	19
3.6.3	Server-Side Deanonymisation and Tool Execution	19

4	Implementation Details	21
4.1	Technology Stack and Development Environment	21
4.2	The Reversible Anonymisation Pipeline	21
4.2.1	Early Approach: Microsoft Presidio and its Limitations	21
4.2.2	The Pivot to GLiNER for Multilingual Italian NER	21
4.2.3	Deterministic Token Mapping and Ephemeral Context Storage	21
4.3	Exposing Business Operations via MCP	21
4.3.1	C# Attributes and Reflection for Tool Schema Generation	21
4.3.2	The <code>ask_antares</code> Routing Tool	21
4.3.3	Strict Prompting for Privacy Enforcement	21
4.4	Use Case Orchestration in <code>JobTools</code>	21
4.4.1	Use Case A: Nearest Workers to a Cantiere	21
4.4.2	Use Case B: Colleagues of a Person	21
4.4.3	Use Case C: Holiday Request Analysis	21
4.5	Backend Integration and Data Abstraction	21
4.5.1	The <code>IMiorelliService</code> Interface	21
4.5.2	The Stub/Mock Pattern	21
4.5.3	NSwag Client Generation and the Migration Path	21
5	Use Cases and Validation	23
5.1	Workforce Management Domain Overview	23
5.2	Use Case A: Spatial Querying for Construction Sites	23
5.3	Use Case B: Colleague Identification and Proximity Ranking	23
5.4	Use Case C: Holiday Request and Absence Analysis	23
6	Discussion and Future Work	25
7	Conclusions	27
7.1	Summary of the Work	27
7.2	Final Remarks on Enterprise AI Adoption	27
	Glossary	27
	Nomenclature list	27
	Bibliography	30
	List of Figures	31
	List of Tables	33

Chapter 1

Introduction

1.1 Context and Motivation

Large language models have demonstrated strong capabilities in interpreting user intent and routing natural-language requests to the right downstream action, capabilities that make them an attractive interface layer for enterprise software. Adoption in real business workflows has lagged behind that promise, and the reason is not technical performance. It is that most enterprise data, personnel records, financial figures, client information, is protected by data protection law and internal policy in ways that make sending it to an external, third-party model provider unacceptable by default. The gap between what LLMs can do and what enterprises are willing to let them see is where this thesis sits.

That gap is concrete, not abstract, in workforce management platforms. Antares/Miorelli is an ERP system used across the Italian facility management sector to manage personnel assignments to client sites, absence and leave requests, and the reporting relationships between workers and their managers. Users interact with it through structured web forms: selecting a site from a dropdown, filtering a worker list, filling in a ticket type. An LLM could replace that friction with a single natural-language question, interpreted directly and mapped onto the same underlying operations, without a rule engine or a trained classifier standing in between. But every one of those operations touches personnel data, and routing the request through a commercial LLM API means, on the face of it, sending that data outside the company’s boundary.

This thesis was carried out as a professional internship at Delta Informatica / Saidea / Diventa S.r.l., an ICT services company based in Trento, within its Research and Development team, and it uses the company’s Antares AI project as the concrete setting in which to work out an answer. The rest of this chapter states the problem precisely and the objectives that follow from it.

1.2 The Problem of Data Privacy in LLM Integration

The data behind every Antares/Miorelli request is personnel-sensitive by construction: worker names, phone numbers, residential coordinates, absence records, and the reporting relationships between employees and managers. All of it is personal data under the GDPR. Sending a raw user prompt such as *“Trova i colleghi di Giulia Conti”* to an external LLM provider means transmitting a real name, and by extension a real person’s employ-

ment relationships, outside the company’s infrastructure and into a third-party service the company does not control. Management set an explicit constraint at the start of the project: no such data may leave the company’s boundary in a form an external party could read.

That constraint rules out the simplest integration path. It does not rule out LLM-based routing itself, but it forces a specific architectural question: how a model can classify the intent of a request and extract its parameters without ever seeing the values those parameters refer to. The chapters that follow show that the answer is not to avoid PII exposure through policy or through trust in a vendor’s data handling terms, but to make exposure structurally impossible: replace every identifying value with a reversible token before the request reaches the model, and restore the original values only inside the company’s own trusted boundary, at the point where the business operation actually executes.

Chapter 2 shows that existing tools, such as rule-based detectors and one-way redaction models, do not yet close this gap.

1.3 Research Objectives and Questions

The central question this thesis investigates is how an external LLM can be integrated as a semantic router without exposing sensitive business data, while still guaranteeing deterministic, verifiable execution of the underlying business logic. Answering it inside a real deployment setting means meeting three conditions at once: the LLM must handle the semantic interpretation of natural-language workforce requests, no personally identifiable information may cross into the model’s context under any circumstance, and the results produced must still be operationally correct, results a human operator could act on without a second, manual verification pass. Those three conditions break down into the more specific questions addressed in the chapters that follow.

The first question is architectural: what division of responsibility between client, anonymisation layer, LLM, and business-logic server makes PII exposure structurally impossible rather than merely policy-forbidden. The second concerns the detection layer itself: whether a zero-shot, multilingual named-entity recognition model can reach the recall required for real Italian-language business text, where the rule-based alternative had already been tried and had failed. The third is about consequences: whether an architecture built around reversible anonymisation can still deliver accurate results on real use cases, or whether the privacy layer costs something in correctness.

1.4 Contributions

This thesis makes three concrete contributions, positioned against the state of the art reviewed in Chapter 2: rule-based detectors and one-way redaction models each solve part of the PII-exposure problem, but neither preserves a path back to the real data once a routing decision has been made.

A privacy-by-design MCP architecture. It presents a four-service system, an anonymisation API, a GLiNER-based NER microservice, an MCP server, and an MCP client, structured so that the single architectural invariant (no PII crosses the LLM boundary in cleartext) is enforced by the topology of the system rather than by a policy the model is merely asked to follow.

A reversible tokenisation pipeline. It describes the design and the reasoning behind a token-based anonymisation scheme that preserves full operational correctness while allowing controlled de-anonymisation at the protocol boundary, including the decision to replace

an initial Presidio-based implementation with a GLiNER-based one after Presidio’s recall on Italian business text proved insufficient for the company’s real use cases.

Empirical validation on real workforce operations. It reports the implementation and end-to-end validation of three business use cases, nearest-worker lookup, colleague identification, and holiday request analysis, each exercised against a realistic mock dataset that reproduces the complexity of the production platform.

Validation throughout this thesis is carried out against a realistic mock data provider rather than the production Miorelli backend, a scope decision made jointly with the host company to prioritise the correctness of the privacy layer within the internship timeframe.

1.5 Structure of the Thesis

The rest of the thesis follows the conventional structure of an engineering thesis [8]: prior work motivates and constrains the design, the design is described in full before it is validated, and the results are discussed critically before drawing conclusions. Chapter 2 reviews the literature this work builds on: LLM integration into enterprise workflows, the Model Context Protocol, and privacy-preserving NLP, closing with the state-of-the-art gap that motivates the architecture. Chapter 3 presents that architecture in detail, from the privacy-by-design principles that govern it to the four-service runtime topology and the lifecycle of a single request. Chapter 4 describes the concrete implementation: the technology stack, the anonymisation pipeline’s internals, the MCP tool layer, and the mock-backed Miorelli integration. Chapter 5 documents the three use cases and the validation carried out against them. Chapter 6 discusses the results, the design trade-offs made along the way, and the work left for a future iteration. Chapter 7 closes the thesis with a summary of what was achieved against the objectives stated in Section 1.3.

Chapter 2

Background and Related Work

This chapter reviews the three bodies of literature that inform the design of the system presented in subsequent chapters: the integration of large language models into enterprise software, the Model Context Protocol as a standardisation layer for LLM tool interfaces, and privacy-preserving natural language processing with a focus on named entity recognition and PII protection. Section 2.1 examines how large language models change semantic routing and tool invocation in enterprise systems, and the engineering constraints that shift introduces. Section 2.2 turns to the Model Context Protocol and the security boundaries it defines between client, server, and model. Section 2.3 addresses the regulatory and technical grounding of PII protection, from the GDPR’s definition of personal data to detection approaches ranging from rule-based systems to zero-shot transformers. Section 2.4 surveys the current state of the art in secure LLM orchestration and identifies the gap that motivates this work.

2.1 Large Language Models in Enterprise Workflows

The modern generation of large language models traces its architectural lineage to the transformer, introduced by Vaswani et al. [23] in 2017. The attention mechanism, which allows each token in a sequence to attend to every other token, proved to be the basis for a qualitative leap in language understanding. Scaling transformer-based models, exemplified by GPT-3 [3] with its 175 billion parameters, showed that sufficiently large models exhibit capabilities not present in smaller counterparts: multi-step reasoning, few-shot generalisation from in-context examples, and reliable instruction following across diverse domains [24].

For enterprise software, the practical implication is a new paradigm for human-system interaction. Until recently, bridging the gap between a user’s natural language expression and a system’s formal data operations required rule engines, supervised intent classifiers, or custom NLP pipelines. A general-purpose language model can close that gap from the prompt alone, interpreting natural language requests and mapping them to well-typed business operations without labelled training data or domain-specific engineering.

The shift creates real engineering problems. Enterprise workflows demand deterministic, auditable outputs. Sensitive business data must not be transmitted to third-party model providers. System behaviour must remain predictable under adversarial or malformed inputs. These constraints sit in direct tension with the generative, probabilistic nature of LLMs, and reconciling them is an active area of research and engineering.

2.1.1 Semantic Routing and Intent Classification

Semantic routing refers to interpreting a natural language utterance and directing it to the appropriate downstream component based on its inferred intent. Classical approaches used supervised intent classifiers trained on labelled utterance datasets [13]. These models required curated training data per domain and were brittle under paraphrasing, multilingual input, or out-of-distribution requests. In practice, that limitation has a direct cost: adding a new business action, or supporting a new language, means collecting and labelling a fresh batch of example utterances before the classifier can be retrained and redeployed, a cost that scales with every intent and every language a deployment needs to cover.

LLMs remove that cost. Because they acquire broad linguistic knowledge during pre-training, they can classify intent zero-shot given only a natural language description of the available actions, with no labelled dataset and no retraining step. Routing logic becomes a prompt rather than a trained artefact, and adding a new action or language is a matter of editing text, not collecting data. A recent example, though outside classic intent classification, is a conversational, RAG-based assistant for AI-supported document creation workflows [6], which replaces a rule-based document generation flow with natural-language interaction end-to-end. The saving is not free, however. The quality of zero-shot routing depends heavily on the specificity and clarity of tool descriptions, and vague or overlapping descriptions cause the model to route incorrectly regardless of the underlying model’s capability, making prompt engineering a first-class concern in deployed systems.

2.1.2 Tool Calling and AI Agents

The capacity for LLMs to invoke external functions (also called function calling) was formalised as a first-class API feature by major model providers in 2023. In this paradigm, the developer supplies the model with a schema describing callable functions, their parameters and return types. The model produces a structured call rather than free text; the host application executes it and feeds the result back for the model to incorporate into its response.

The ReAct framework [25] formalised the interleaving of reasoning traces and tool invocations, showing that a model that explicitly reasons about which tool to call outperforms models that invoke tools without intermediate reasoning, especially in multi-step tasks. This is the AI agent paradigm: a language model acting as an orchestrator that selects and sequences tool calls to reach a higher-level goal.

The Toolformer work [22] demonstrated that models can learn to use external APIs from self-supervised examples, acquiring tool use directly from patterns in text rather than from an explicit interface definition. Nothing in that training signal constrains the resulting call to a fixed, machine-checkable format: a malformed or incomplete invocation is not distinguishable, from the model’s own output, from a well-formed one. That gap is not cosmetic in an enterprise setting. A routing decision that cannot be validated against a schema cannot be logged in a structured, replayable form, and there is no reliable way to confirm after the fact which tool was called with which parameters. Later systems such as Gorilla [19] narrowed the correctness gap by fine-tuning a model on API documentation and evaluating calls by execution rather than surface similarity, but the underlying call is still free text scored after the fact, not structurally guaranteed beforehand. For this reason, enterprise deployments generally prefer explicit schema-driven invocation over learned tool use. Schema-driven invocation provides stronger output format guarantees and makes the routing decision auditable.

Two architectural properties of tool-calling systems have direct implications for enterprise deployments. First, the model’s agency can be bounded by restricting which tools it may call: exposing only business-level operations and keeping data access inside deterministic code limits the model to intent classification and parameter extraction. Second, the tool result feedback loop, in which tool outputs are returned to the model for further reasoning, is the primary vector for indirect prompt injection attacks [10], making its design a security-relevant decision.

2.2 The Model Context Protocol (MCP)

The Model Context Protocol [2], published by Anthropic in November 2024, standardises the communication interface between LLM client applications and external tool servers. Before MCP, each LLM-tool integration required bespoke glue code, custom serialisation formats, and proprietary conventions. MCP defines a vendor-neutral interface so that any compliant client can connect to any compliant server without integration-specific adaptation.

The design is modelled after the Language Server Protocol (LSP) [4], which eliminated the $N \times M$ explosion of editor-language integrations by decoupling language-analysis servers from editor clients. MCP applies the same logic: any client that speaks MCP can talk to any server that speaks MCP, reducing the integration surface from $N \times M$ to $N + M$.

2.2.1 Standardizing LLM Interfaces

MCP defines three categories of server-exposed primitives [2]. *Tools* are callable functions with typed input schemas and structured return values, analogous to OpenAPI operations. *Resources* are read-only data sources the model can pull into its context: file contents, database records, configuration objects. *Prompts* are reusable, parameterised message templates.

Communication runs over two transports: standard input/output for locally spawned server processes, and HTTP with Server-Sent Events (SSE) for network-accessible servers. The underlying message format is JSON-RPC 2.0. Tool schemas are JSON Schema objects describing parameter types, constraints, and documentation. This standardised schema representation allows tooling to auto-generate type-safe client code from server definitions, reducing the risk of schema drift between client expectations and server implementation.

2.2.2 Security Boundaries in MCP

MCP’s security model separates three roles. The *MCP host* manages client connections and controls which servers the model may reach. The *MCP server* is a trusted executor with access to real data and infrastructure. The *model* is a semantic reasoner: it produces structured tool calls but accesses no data source directly.

This three-layer separation creates a natural enforcement point for privacy. The server is the only component that handles raw sensitive data. A pre-processing step applied to user requests before they reach the model can structurally prevent PII from entering the LLM context, because the server’s trust boundary sits downstream of the model. The model operates only on what the client chooses to send.

Prompt injection [10] is the primary known attack on this architecture. Malicious content embedded in tool results or externally retrieved resources can attempt to hijack the model’s behaviour by simulating system-level instructions. Mitigations include restricting the model’s tool set, limiting the scope of requests the model may issue, and routing business

logic through deterministic code that does not re-enter the model after execution. These mitigations reduce the attack surface but do not eliminate it, and the design of the tool result feedback loop remains an open security concern in MCP deployments. A systematic security analysis of the MCP ecosystem [12] catalogues these risks across the full server lifecycle, from malicious tool registration to compromised transport, and finds that few real deployments implement the safeguards it recommends.

2.3 Privacy-Preserving Natural Language Processing

User requests directed at business systems routinely contain personally identifiable information: employee names, site addresses, phone numbers, absence dates. Under data protection regulations, transmitting this information to external model providers requires a valid legal basis and appropriate safeguards. This section reviews the regulatory context and the technical approaches developed to address PII exposure in NLP pipelines.

2.3.1 Personally Identifiable Information (PII) Definitions

The General Data Protection Regulation [9], Article 4(1), defines personal data as “any information relating to an identified or identifiable natural person.” Identifiability is broad: a person is identifiable if they can be singled out by name, identification number, location data, or factors specific to their physical, professional, or social identity. Combinations of individually innocuous data points can identify an individual in aggregate.

Article 4(5) introduces *pseudonymisation*: processing personal data so that it can no longer be attributed to a specific person without additional information, provided that additional information is kept separately with appropriate technical safeguards. Pseudonymisation is distinct from anonymisation. Pseudonymised data remains personal data under the GDPR, because the mapping between pseudonyms and identities is preserved, even if stored separately. Truly anonymous data cannot be re-linked by any reasonably likely means and falls outside the regulation entirely.

This distinction has direct engineering implications for privacy-preserving LLM pipelines. A system that replaces PII with reversible tokens before sending data to an external model provider is performing pseudonymisation, not anonymisation. The privacy guarantee it provides is a limitation of PII exposure to the model, not its elimination. The token-to-entity mapping must itself be protected and handled as personal data under the GDPR.

2.3.2 Named Entity Recognition (NER) for Data Masking

Named entity recognition is the NLP task of identifying and classifying contiguous spans of text that refer to named entities: persons, organisations, locations, dates, phone numbers. For PII protection, NER is the detection layer of a two-stage pipeline: entities are identified, then replaced with neutral placeholders that preserve syntactic structure without revealing identifying content.

Classical NER systems based on Conditional Random Fields or bidirectional LSTMs required annotated corpora per language and domain. Pre-trained transformer encoders, BERT [7] and its multilingual variants, reduced data requirements substantially by enabling fine-tuning on small labelled datasets. Domain and language specificity remained a persistent challenge. A model trained on English newswire won’t reliably handle Italian business communications.

The error asymmetry in masking pipelines is architecturally significant. A false negative (a missed PII span) directly violates the privacy guarantee: the unmasked entity crosses the model boundary in plaintext. A false positive (a masked non-PII span) degrades the utility of the downstream task but creates no privacy breach. This asymmetry implies that detection components should be optimised for recall, with post-processing to suppress common false positives.

The design of replacement tokens also affects downstream task quality. Replacing every entity with a single generic token (e.g., [REDACTED]) destroys structural information the model may need: if a request mentions two different people and both become the same token, the model cannot distinguish them when making a routing decision. Label-typed, numbered tokens such as <PERSON_0001> and <PERSON_0002> preserve entity type and cardinality while removing identifying content, enabling correct multi-entity routing even when all PII has been replaced.

2.3.3 Rule-based vs. Deep Learning Approaches (Presidio vs. GLiNER)

Two broad approaches are available for PII detection in NLP pipelines: rule-based systems that combine regular expressions, gazetteers, and lightweight statistical models; and transformer-based systems that apply encoder models to span classification over arbitrary text.

Microsoft Presidio [16] is the leading open-source rule-based framework. It provides recognisers for names, phone numbers, email addresses, credit card numbers, and national identifiers, built on regular expressions and context-sensitive matching, optionally extended with spaCy NER models. Its strengths are predictability, low computational cost, and an extensible plugin architecture. Its weaknesses are language dependence (most recognisers target English) and rigidity: entity types that don't match predefined patterns are silently missed, with no indication of failure. Extending coverage to a new language or entity type requires writing explicit rules, which is labour-intensive and inherently incomplete.

GLiNER [26] (Generalist Model for Named Entity Recognition) is a zero-shot NER model built on a bidirectional transformer encoder. Instead of training a fixed classification head per entity type, GLiNER conditions detection on natural language label descriptions given at inference time. The model computes a similarity score between the label description and each candidate span, enabling detection of arbitrary entity types without fine-tuning. The multilingual variant `urchade/gliner_multi-v2.1`, trained on data spanning multiple European languages, provides coverage of morphologically rich languages such as Italian, including proper name patterns and domain-specific vocabulary that rule-based approaches miss.

Compared to Presidio, GLiNER offers three concrete advantages for multilingual and domain-diverse deployments. The detection label set is configurable at runtime without model retraining. Italian-language support is native, not an adaptation. Each prediction carries a confidence score, enabling a configurable threshold to tune the precision-recall trade-off. A post-processing pipeline over raw GLiNER predictions (span normalisation, stopword filtering, overlap deduplication) can further improve precision without sacrificing recall on genuine PII.

2.4 State of the Art in Secure LLM Orchestration

The dominant pattern in privacy-sensitive LLM deployments is a *privacy-preserving proxy*: a pre-processing layer that sanitises user input before it reaches the model. Early implemen-

tations used regular expression substitution, adequate for structured PII (phone numbers, email addresses) but inadequate for unstructured natural language. More recent work applies transformer-based NER, trading computational cost for substantially better recall on diverse inputs.

Three systems anchor the current state of the art for PII detection in LLM pipelines: Microsoft Presidio, the rule-based baseline discussed in Section 2.3.3; GLiNER, the zero-shot transformer detector adopted in the architecture presented in this thesis; and OpenAI’s Privacy Filter, reviewed in detail below as the most complete open-weight alternative. A related concern shows up in adjacent academic work: Mansillo [14] anonymises synthetic training data inside a multimodal retrieval-augmented pipeline, which confirms that anonymisation-aware LLM systems are an active area of interest beyond this thesis, though that work does not address reversibility or tool-execution correctness either. Two recent papers push in the same direction from different angles. Hou et al. [11] propose a general pseudonymisation framework for cloud-based LLMs that replaces private information before a remote API call, evaluated for privacy and utility but not for downstream tool-execution correctness. Albanese et al. [1] anonymise text on-premise with a locally run LLM before it reaches a question-answering agent, prioritising factual utility over the reversibility this thesis requires. The three systems diverge along the dimensions that matter for an enterprise LLM orchestration system, and no single one of them covers all of them.

All three run entirely on local infrastructure and ship under a permissive licence, MIT for Presidio and Apache 2.0 for the other two, so neither dimension separates them.

Multilingual coverage is uneven, however. Presidio’s recognisers are English-centric and need new rules for each additional language, GLiNER’s multilingual variant handles Italian natively, and Privacy Filter’s non-English performance is undocumented and, per its own model card, may degrade on morphologically rich languages. Reversibility separates them further. Presidio can be reversible in principle through a dedicated encrypt/decrypt operator pair, but that ties reversibility to key management external to the detection pipeline. GLiNER is a detector with no built-in notion of a token-to-entity mapping; reversibility, where it exists, is a property of the system built around it, not of GLiNER itself. Privacy Filter performs one-way redaction by design and discards the mapping entirely. On detection accuracy, Privacy Filter is the only one of the three to publish a comparable accuracy figure, discussed in detail in Section 2.4.1. Presidio and GLiNER have no comparably reported number. Presidio is tuned for precision over broad recall by construction, and GLiNER is tuned for recall across arbitrary label sets, so neither figure would be comparable to Privacy Filter’s self-reported one even if it existed.

None of the three systems combines reversibility with a validated benchmark for operational correctness. The one system with a quantitative accuracy figure, Privacy Filter, is also the one that discards the token-to-entity mapping outright. The systems that could in principle preserve or approximate that mapping, Presidio and GLiNER, report no comparable benchmark of their own.

The literature addresses detection and masking largely in isolation. End-to-end systems that detect PII, mask it before the model boundary, execute business operations using real values server-side, and return a safe response to the client remain uncommon in published work. Most published systems stop at the masking step and do not address the operational requirement that tool execution needs the original entity values, not their placeholders.

2.4.1 OpenAI Privacy Filter: A Reference Open-Weight Model for PII Redaction

The most significant open-weight contribution to this space is OpenAI’s Privacy Filter [18], released in April 2026 under the Apache 2.0 licence. It is the first publicly available, production-quality PII detection model explicitly designed as a pre-processing layer for LLM pipelines.

Architecture. Privacy Filter is a bidirectional token-classification model derived from the GPT OSS family. The autoregressive generation head is replaced by a classification head over a BIOES span-labelling scheme with 33 output classes. A constrained Viterbi decoder enforces coherent span boundaries across multi-token entities. The Sparse Mixture-of-Experts design gives 1.5 billion total parameters but only approximately 50 million active per token, enabling high-throughput inference at low cost.

Capabilities. The model detects eight PII categories: `private_person`, `private_address`, `private_email`, `private_phone`, `private_url`, `private_date`, `account_number`, and `secret`. Its context window is 128,000 tokens. On the PII-Masking-300k benchmark it achieves 96% F1 (97.43% on the corrected version). Fine-tuning on small in-domain datasets can push domain-specific performance from 54% to 96% F1 quickly, suggesting a strong general-purpose foundation.

Local execution. The model is designed to run on-premise, within the organisation’s infrastructure, removing the need for a Data Processing Agreement with a cloud vendor and addressing GDPR data residency requirements directly.

One-way redaction. Privacy Filter performs one-way redaction: the placeholder tokens it produces carry no mapping back to the original entities. This is appropriate for use cases where the model’s output does not need to reference original PII values, such as summarisation, classification, and general question-answering. For enterprise workflows in which a routing decision must trigger operations on real, identified data, one-way redaction is insufficient. A system that routes a request mentioning `<LOCATION_0001>` must know what `<LOCATION_0001>` is at the point of tool execution.

OpenAI’s model card acknowledges that Privacy Filter performs pseudonymisation under GDPR Article 4(5), not anonymisation. The placeholder tokens combined with the original document could in principle be reversed by a party holding both. This is the correct framing of what such systems achieve: they limit PII exposure to the model provider, but the mapping between tokens and entities still exists and must be managed.

Multilingual limitations. Privacy Filter’s performance on non-English languages is not officially documented and may degrade significantly on morphologically rich languages such as Italian.

The body of work reviewed in this chapter establishes that privacy-preserving pre-processing is the right pattern for LLM orchestration in sensitive enterprise domains. What the existing systems do not address is the combination of reversibility with operational correctness: enabling a routing system to work on pseudonymised inputs while guaranteeing that tool execution uses the original entity values, without exposing those values to the model provider. This gap is the starting point for the architecture described in Chapter 3.

Chapter 3

System Architecture and Design

This chapter presents the architecture of Antares AI, a privacy-preserving LLM orchestration system for enterprise workforce management. Chapter 1 and Chapter 2 set out the problem this architecture answers: an LLM has to route natural-language workforce requests without ever seeing the personal data those requests refer to. The design that follows is organised around one central invariant, no personally identifiable information crosses the LLM context boundary in cleartext, and most of the decisions described in this chapter follow from what it takes to hold that invariant in practice. Section 3.1 sets out the privacy-by-design principles that govern every subsequent decision. Section 3.2 introduces the Antares/Miorelli domain and the constraints it places on the system. Section 3.3 translates those constraints into a traceable list of functional and non-functional requirements. Section 3.4 presents the four-service runtime architecture. Section 3.5 details the design of each component. Section 3.6 traces a complete request through the system end to end.

3.1 Architectural Principles: Privacy-by-Design

Privacy by design, formalised by Cavoukian [5] and later codified as a legal obligation in GDPR Article 25 [9], is the principle that privacy protection must be built into systems from the outset. Cavoukian’s framework identifies seven foundational principles: proactive prevention of privacy harms, privacy as the default operational setting, privacy functionality embedded into the architecture, full functionality without zero-sum trade-offs, end-to-end security across the data lifecycle, visibility and transparency in system operation, and user-centric design. The Antares AI architecture applies these principles as structural design constraints, not as runtime policy.

Three of Cavoukian’s principles shape the system most directly.

Privacy as the default. Anonymisation of user requests before LLM transmission is the only supported operational path. A client that skips the anonymisation step sends an unprocessed prompt, but the server system prompt still instructs the model to treat input as pseudonymised tokens; bypassing anonymisation is detectable and requires an explicit decision, not an oversight. Nothing in the system’s configuration surface offers a way to switch privacy enforcement off.

Privacy embedded into design. PII isolation is not achieved through audit logging, access controls, or post-hoc output redaction. It is structural: by placing the anonymisation service at the architectural boundary, PII is physically absent from the data that crosses it, so there is nothing for the model to leak in the first place.

End-to-end security. The token-to-entity mapping is stored in an ephemeral, session-scoped Redis context and consumed exactly once during server-side deanonymisation. No component persists PII beyond the minimum lifetime required for a single business operation. Operational reports written to disk by the `JobTools` layer stay on server-local storage and are never sent to the client.

Together these constraints define the *architectural invariant*: PII may exist in the system, but only within the server’s trusted execution boundary, and only for the duration of one business operation.

3.2 The Antares AI Ecosystem

Antares AI was developed within the Antares/Miorelli ecosystem, an ERP platform for workforce management used in the Italian facility management sector. The platform manages personnel assignments to job sites (*cantieri*), tracks absence and leave requests, and maintains the hierarchical relationships between workers and managers. Users currently interact via structured web interfaces; Antares AI enables the same operations through natural language, routing each request to the correct business tool automatically.

The domain is demanding on privacy and correctness for concrete reasons. The data is personnel-sensitive by nature: names, phone numbers, residential coordinates, absence records, and employment relationships are all PII under the GDPR. The user base is multilingual (requests arrive in Italian, regional dialects, or a mix), which ruled out recogniser-based NER tools designed for English. Most critically, the operational context requires accurate results: a holiday request analysis that produces an incomplete result is operationally harmful, not just inconvenient, because it may leave a client site under-staffed. The system must preserve full business logic accuracy even when anonymising inputs.

Workforce operations are structured around three core entity types: *persone* (workers), *cantieri* (client sites), and *ticket* (absence and leave records). Relations are typed: `PULITO_DA` links a worker to their assigned sites, `RESPONSABILE_DI_RIFERIMENTO` identifies a worker’s manager, and `AUTORIZZATORE` identifies the person authorised to approve leave. These entity types and relation names appear in natural language requests and define the PII surface the anonymisation pipeline must cover.

3.3 Functional and Non-Functional Requirements

The principles and domain constraints introduced above translate into a concrete set of requirements the architecture must satisfy. This section makes them explicit and traceable, without repeating the discussion already given in Sections 3.1 and 3.2.

Functional requirements follow directly from the three use cases the system supports, each one a concrete operation a dispatcher or site manager already performs by hand. Given a named client site, the architecture has to find the nearest available workers within a configurable radius and rank them by distance (Use Case A, traced in detail in Section 3.6). Given a worker’s name, it has to identify that worker’s direct colleagues and surface a ranked pool of nearby substitutes (Use Case B, Section 4.4.2). Given a holiday request, it has to resolve the full chain end to end: verify the requester’s authorisation, determine which colleagues are absent on the relevant dates, and identify who is available to cover (Use Case C, Section 4.4.3).

Non-functional requirements follow from the domain constraints described in Section 3.2, and each one is really a boundary condition on the functional requirements above

rather than an independent property. The privacy invariant from Section 3.1 sets the hard limit: no PII may cross the LLM context boundary in cleartext, under any request pattern. Operational accuracy sets a second one, in the opposite direction, since a routing or entity-resolution error introduced by the privacy layer is not an acceptable trade-off when an incomplete result can leave a client site under-staffed. Multilingual support is the third: the system has to handle Italian as a native language, not as an adaptation of an English-first pipeline, given how the actual user base writes.

3.4 High-Level System Architecture

Antares AI comprises four cooperating runtime services, each with a distinct responsibility and a well-defined interface. Figure 3.1 illustrates the runtime topology and data flows.

The four services and their roles are summarised in Table 3.1.

Table 3.1: Antares AI runtime services

Service	Stack	Port	Role
MCP Client	C#/.NET 10	—	Request orchestration and test harness
Anonymization API	C#/.NET 10	5292	PII detection, tokenisation, context storage
GLiNER Service	Python/FastAPI	5100	Named entity recognition
MCP Server	C#/.NET 10	5001	LLM routing and business tool execution

The communication topology mirrors the system’s trust model. The MCP client is an untrusted endpoint: it never receives real personnel data, and the only thing it sees is a hardcoded Italian confirmation string. The anonymisation API sits at the client-side boundary, processing the raw prompt and issuing a session token (`contextId`) that indexes the stored mapping. The GLiNER service is a stateless NER oracle; it knows nothing about tokens, Redis, or business context. The MCP server is the trusted execution boundary: it receives only anonymised prompts, routes them via Claude, restores PII server-side, and executes business tools against real data.

Redis [21] runs at port 55000 as the ephemeral context store. It is not reachable from the client and is accessible only to the anonymisation API (writing) and the MCP server (reading and consuming).

The architectural invariant (no PII crosses the MCP boundary) is enforced at the interface between client and server. The `ask_antares` tool accepts an anonymised prompt and a `contextId`; it never receives raw PII. The client calls the anonymisation API first, gets an anonymised prompt and a `contextId`, and only then constructs the tool call. PII restoration from the `contextId` happens inside the server’s trusted boundary, after the routing decision has been made.

3.5 Component Design

3.5.1 The MCP Client Orchestrator

The MCP client (`Antares_AI_McpClient`) is a .NET 10 [15] console application that orchestrates the end-to-end request lifecycle. In the current system it serves as both the reference client implementation and the integration test harness for the three use case scenarios.

Its responsibilities are deliberately minimal. It constructs a natural language prompt, submits it to the Anonymization API, and then calls `ask_antares` on the MCP server with

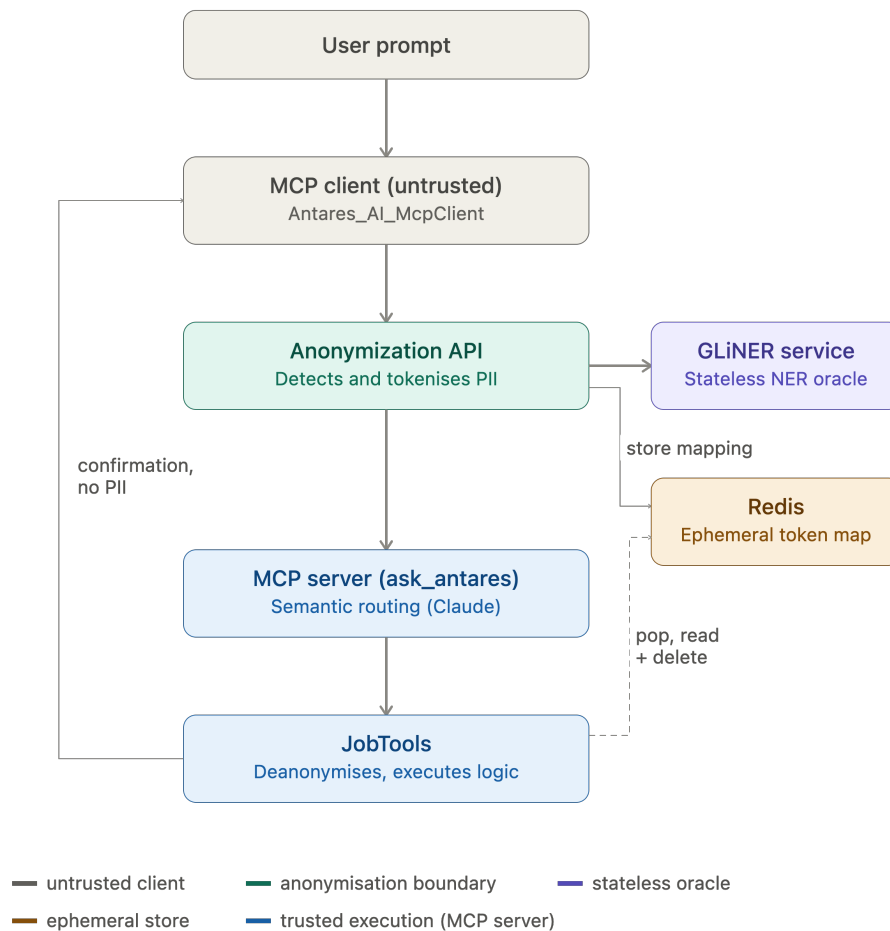


Figure 3.1: Runtime architecture of Antares AI. The client anonymises the user prompt before crossing the MCP boundary. The MCP server routes the anonymised request via Claude and executes the selected business tool server-side, where PII is restored from the Redis context. The client receives only a hardcoded confirmation string.

the anonymised prompt and `contextId`. The client performs no business logic and holds no sensitive data; it can't observe PII because it never calls a deanonymisation endpoint or touches the Redis context.

Communication with the MCP server uses the HTTP/SSE transport via `ModelContextProtocol.Core`. Communication with the Anonymization API uses a strongly typed HTTP client generated by NSwag [17] from the API's OpenAPI specification, keeping the client's view of the API interface in sync with the server implementation automatically.

A production client would accept dynamic user input, manage multi-turn context across `contextId` sessions, and surface token placeholders in the UI as opaque references. The current implementation uses hardcoded prompts to enable deterministic integration testing.

3.5.2 The Anonymization Service API

The Anonymization Service (`AnonymizationServiceAPI`) is an ASP.NET Core Minimal API application that acts as the PII gateway between client and server. It exposes three REST endpoints: `GET /anonymize`, `POST /anonymize-into-context`, and `POST /deanonymize-known`.

The primary client-facing operation is `/anonymize`. It receives a raw prompt, delegates entity detection to the GLiNER service, replaces detected entities with stable typed tokens, generates a `contextId`, stores the token-to-entity mapping in Redis, and returns the anonymised text, the `contextId`, and a detection count. The token format is `<LABEL_NNNN>`, where `LABEL` is the uppercase entity type and `NNNN` is a zero-padded counter per label per context. This preserves entity type and cardinality while stripping identifying content.

The `/anonymize-into-context` endpoint extends an existing session by merging new token mappings into an already-issued `contextId`. This supports multi-turn interactions where a follow-up request introduces a new entity that must be pseudonymised within the same operational context.

The `/deanonymize-known` endpoint is the server-facing operation. It receives an anonymised string and a `contextId`, retrieves the mapping from Redis via a `Pop` operation (atomic read-and-delete), and restores original values through string substitution. The one-time-use design limits the exposure window for the mapping to the minimum needed for tool execution; a repeated call with the same `contextId` will find no context and fail cleanly.

Detection is fully delegated to the GLiNER service. The separation between detection (GLiNER) and masking plus context management (Anonymization API) preserves substitutability: the NER backend can be replaced without touching the tokenisation logic.

3.5.3 The GLiNER-based NER Microservice

The GLiNER service (`GLiNERService`) is a Python FastAPI [20] application that exposes a single stateless endpoint, `POST /analyze`, and a health check at `GET /health`. Its job is to apply the GLiNER model to an input text and return a list of detected entity spans with labels, character offsets, and confidence scores.

The multilingual model `urchade/gliner_multi-v2.1` [26] is loaded once at startup and held in memory for the process lifetime. Model weights download from HuggingFace on first run and are cached locally; subsequent starts use the cached copy.

The service is a dumb NER oracle. It receives text and labels, runs the model, returns spans. All policy decisions (which labels to request, what confidence threshold to apply, how to handle overlapping spans) are made by the Anonymization API, which configures the `/analyze` call accordingly. This keeps the GLiNER service simple and reusable.

Three post-processing stages run within the Anonymization API before token substitution. Raw predictions are normalised: invalid offsets, missing labels, and out-of-bounds spans are discarded. Low-quality entities are filtered out: spans below the 0.45 confidence threshold, spans shorter than three characters, and spans matching an Italian stop-word blocklist are removed. Overlapping predictions are deduplicated, keeping the higher-confidence, longer span. The result is a clean, non-overlapping list of entity spans ready for replacement.

3.5.4 The Antares MCP Server

The MCP server (`Antares_AI_McpServer`) is the trusted execution boundary. It exposes the `ask_antares` routing tool, calls the Claude API for semantic routing, and executes the selected business tool against the Miorelli data layer.

Tools are registered via `ModelContextProtocol.AspNetCore`, which generates MCP-compliant schemas from C# class and method annotations. Two types are registered: `AntaresTools` (exposing `ask_antares`) and `JobTools` (exposing the three business operations). The `JobTools` methods are not called directly by the MCP client; Claude invokes them through function-calling within an `ask_antares` execution.

The server calls Claude Sonnet 4.6 via the `Anthropic.SDK` client. The system prompt defines Claude’s role as a semantic router, lists the available sub-tools with descriptions, and enforces privacy constraints: the model must not infer or reveal PII from tokens, must forward the `contextId` to the selected sub-tool, and must include the sub-tool’s result string verbatim in its response. A retry loop with exponential backoff handles transient `overloaded_error` responses, retrying up to four times before returning an error message.

The data access layer is abstracted behind `IMiorelliService`. The current implementation registers `MiorelliServiceStub`, which throws `MiorelliClientNotAvailableException` on every call; each business tool catches this and falls back to static mock data from `MiorelliMockDataProvider`. Switching to the real Miorelli tenant requires changing one line in `program.cs`: registering `AntaresMiorelliClient` instead of the stub. No tool code changes. This design is discussed in Chapter 4.

3.6 Data Flow and Lifecycle of a Request

This section traces a user request from natural language prompt to confirmation response. Use Case A (finding the nearest available workers to a named client site) serves as the concrete example.

3.6.1 Tokenisation and Ephemeral Context Storage

The lifecycle begins when the MCP client constructs a natural language prompt. For Use Case A, a typical prompt is: *“Trova i lavoratori più vicini al cantiere Bergamo Via della Fonda 5 Giulia S.r.l. entro 15 km. Risolvi tutti i GUID internamente.”* This contains two PII spans: a location name (*Bergamo Via della Fonda 5*) and a company name (*Giulia S.r.l.*).

The client calls `GET /anonymize`. The API calls the GLiNER service, receives entity predictions, runs the three-stage filtering pipeline, and sorts surviving entities by character offset in reverse order. Replacements are applied right-to-left to avoid invalidating offsets of earlier spans. The location becomes `<LOCATION_0001>` and the company becomes `<ORGANIZATION_0001>`.

The mapping `{"<LOCATION_0001>": "Bergamo Via della Fonda 5", "<ORGANIZATION_0001>": "Giulia S.r.l."}` is stored in Redis under `anon:context:{contextId}`, where `contextId` is a freshly generated UUID. The API returns the anonymised prompt and the `contextId` to the client. No PII is in the response; the client holds only an opaque session identifier.

Redis is the right tool here for practical reasons: its in-memory architecture gives sub-millisecond read and write latency, which matters in a synchronous request pipeline. Its native hash type matches the token-map structure directly, and its in-memory-with-optional-persistence model suits ephemeral session data that does not need to survive a service restart.

3.6.2 Isolated Semantic Routing

The client constructs the `ask_antares` tool call with the anonymised prompt and `contextId`, dispatching it over HTTP/SSE. At this point the MCP boundary has been crossed and no PII is in transit.

The `ask_antares` implementation builds a Claude API call. The system prompt defines Claude's role as an Italian-language semantic router, provides the JSON schemas of the available sub-tools, and specifies the expected response format. The anonymised user prompt becomes the user message.

Claude's task is intent classification (which sub-tool?) and parameter extraction (what values, in anonymised form?). For Use Case A it classifies the intent as `get_nearest_workers_to_cantiere` and extracts `cantiereName = "<LOCATION_0001>"` and `context_id = {contextId}`. It doesn't try to resolve the token; it works entirely with the pseudonymised representation.

The Claude API returns a structured tool call rather than free text. The `Microsoft.Extensions.AI` abstraction intercepts it, locates the registered `GetNearestWorkersToCantiere` method, and invokes it with the extracted parameters. The Redis context is not touched during this phase.

3.6.3 Server-Side Deanonimisation and Tool Execution

The `GetNearestWorkersToCantiere` method is the first point where PII re-enters the system. Its first action is to call `DeanonimizeAsync`, which posts to `/deanonimize-known` with the token `"<LOCATION_0001>"` and the `contextId`. The API atomically reads and deletes the context from Redis and returns the original value: `"Bergamo Via della Fonda 5"`. The Redis context no longer exists after this call.

The deanonymised value drives the business logic entirely within the server's trusted boundary. The method resolves the caller's authorised cantieri via the Miorelli service layer, locates the matching site, retrieves assigned workers via the `PULITO_DA` relation, computes Haversine distances from each worker's registered coordinates to the site's GPS coordinates, and filters and ranks results by distance within the specified radius.

The full report, containing real names, phone numbers, distances, and status information, is written to a timestamped file in the server's local `logs/` directory. It is never transmitted to the client. The method returns a hardcoded Italian confirmation string: *"Operazione completata con successo."* Claude includes this verbatim in its final response.

The client receives a short text block with no PII. The invariant holds across the entire lifecycle: no personally identifiable information crossed the MCP boundary, and a complete, accurate operational result was produced server-side.

Chapter 4

Implementation Details

4.1 Technology Stack and Development Environment

4.2 The Reversible Anonymisation Pipeline

4.2.1 Early Approach: Microsoft Presidio and its Limitations

4.2.2 The Pivot to GLiNER for Multilingual Italian NER

4.2.3 Deterministic Token Mapping and Ephemeral Context Storage

4.3 Exposing Business Operations via MCP

4.3.1 C# Attributes and Reflection for Tool Schema Generation

4.3.2 The ask_antares Routing Tool

4.3.3 Strict Prompting for Privacy Enforcement

4.4 Use Case Orchestration in JobTools

4.4.1 Use Case A: Nearest Workers to a Cantiere

4.4.2 Use Case B: Colleagues of a Person

4.4.3 Use Case C: Holiday Request Analysis

4.5 Backend Integration and Data Abstraction

4.5.1 The IMiorelliService Interface

4.5.2 The Stub/Mock Pattern

4.5.3 NSwag Client Generation and the Migration Path

Chapter 5

Use Cases and Validation

- 5.1 Workforce Management Domain Overview
- 5.2 Use Case A: Spatial Querying for Construction Sites
- 5.3 Use Case B: Colleague Identification and Proximity Ranking
- 5.4 Use Case C: Holiday Request and Absence Analysis

Chapter 6

Discussion and Future Work

Chapter 7

Conclusions

7.1 Summary of the Work

7.2 Final Remarks on Enterprise AI Adoption

Bibliography

- [1] Federico Albanese, Pablo Ronco, and Nicolás D’Ippolito. *Anonymous-by-Construction: An LLM-Driven Framework for Privacy-Preserving Text*. <https://arxiv.org/abs/2603.17217>. arXiv preprint arXiv:2603.17217. Accessed: July 2026. 2026.
- [2] Anthropic. *Model Context Protocol Specification*. <https://modelcontextprotocol.io/specification>. Open protocol for connecting LLM clients to external tools and data sources. Accessed: June 2026. 2024.
- [3] Tom Brown et al. “Language models are few-shot learners”. In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 1877–1901.
- [4] Hendrik Bänder. “Decoupling language and editor — the impact of the language server protocol on textual domain-specific languages”. In: *Proceedings of the 7th International Conference on Model-Driven Engineering and Software Development (MODELSWARD)*. 2019, pp. 129–140. DOI: 10.5220/0007556301290140.
- [5] Ann Cavoukian. *Privacy by Design: The 7 Foundational Principles*. Tech. rep. Information and Privacy Commissioner of Ontario, Canada, 2009.
- [6] Vincenzo Dalia. “Workflow assistito da AI per la creazione documentale: un approccio conversazionale basato su RAG”. Relatori: Luigi De Russis, Daniele Sabetta; sessione di laurea Aprile 2025. Accessed: July 2026. Tesi di Laurea Magistrale in Ingegneria Informatica. Politecnico di Torino, 2025.
- [7] Jacob Devlin et al. “BERT: Pre-training of deep bidirectional transformers for language understanding”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Vol. 1. 2019, pp. 4171–4186.
- [8] Umberto Eco. *Come si fa una tesi di laurea*. La Nave di Teseo Editore spa, 2017.
- [9] European Parliament and Council of the European Union. *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)*. Official Journal of the European Union, L 119, pp. 1–88. 2016.
- [10] Kai Greshake et al. “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection”. In: *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. 2023, pp. 79–90.
- [11] Shilong Hou et al. *A General Pseudonymization Framework for Cloud-Based LLMs: Replacing Privacy Information in Controlled Text Generation*. <https://arxiv.org/abs/2502.15233>. arXiv preprint arXiv:2502.15233. Accessed: July 2026. 2025.

- [12] Xinyi Hou et al. *Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions*. <https://arxiv.org/abs/2503.23278>. arXiv preprint arXiv:2503.23278. Accessed: July 2026. 2025.
- [13] Stefan Larson et al. “An Evaluation Dataset for Intent Classification and Out-of-Scope Prediction”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2019, pp. 1311–1316. DOI: 10.18653/v1/D19-1131.
- [14] Daniele Mansillo. “Multimodal RAG for Slide Presentations with Synthetic Data Generation and Anonymization”. Relatori: Daniele Apiletti, Simone Monaco. Accessed: July 2026. Tesi di Laurea Magistrale in Data Science and Engineering. Politecnico di Torino, 2025.
- [15] Microsoft. *.NET 10*. <https://dotnet.microsoft.com>. Cross-platform application framework used for the MCP server, anonymisation API, and MCP client. Accessed: June 2026. 2025.
- [16] Microsoft. *Presidio — Data Protection and De-identification SDK*. <https://github.com/microsoft/presidio>. Open-sourced by Microsoft in 2018. Accessed: June 2026. 2018.
- [17] NSwag contributors. *NSwag: The Swagger/OpenAPI toolchain for .NET*. <https://github.com/RicoSuter/NSwag>. Used for automatic C# client generation from OpenAPI specifications. Accessed: June 2026. 2020.
- [18] OpenAI. *Model Card for OpenAI Privacy Filter*. <https://cdn.openai.com/pdf/c66281ed-b638-456a-8ce1-97e9f5264a90/OpenAI-Privacy-Filter-Model-Card.pdf>. Released April 22, 2026 under the Apache 2.0 licence. Sparse Mixture-of-Experts token-classification model for PII detection. Benchmark F1 figures (Table 1, Section 7.4.1, p. 14): 96.0% token-level F1 on PII-Masking-300k (baseline), 97.4% on the OpenAI-corrected version. Accessed: July 2026. 2026.
- [19] Shishir G Patil et al. “Gorilla: Large Language Model Connected with Massive APIs”. In: *arXiv preprint arXiv:2305.15334* (2023).
- [20] Sebastián Ramírez. *FastAPI*. <https://fastapi.tiangolo.com>. Modern, high-performance web framework for building APIs with Python. Accessed: June 2026. 2018.
- [21] Redis Ltd. *Redis*. <https://redis.io>. In-memory data structure store used for ephemeral session context storage. Accessed: June 2026. 2009.
- [22] Timo Schick et al. “Toolformer: Language models can teach themselves to use tools”. In: *Advances in Neural Information Processing Systems* 36 (2023), pp. 68539–68551.
- [23] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in Neural Information Processing Systems* 30 (2017).
- [24] Jason Wei et al. “Emergent abilities of large language models”. In: *Transactions on Machine Learning Research* (2022).
- [25] Shunyu Yao et al. “ReAct: Synergizing reasoning and acting in language models”. In: *The Eleventh International Conference on Learning Representations*. 2023.
- [26] Urchade Zaratiana et al. “GLiNER: Generalist model for named entity recognition using bidirectional transformer”. In: *arXiv preprint arXiv:2311.08526* (2023).

List of Figures

3.1 Runtime architecture of Antares AI. The client anonymises the user prompt before crossing the MCP boundary. The MCP server routes the anonymised request via Claude and executes the selected business tool server-side, where PII is restored from the Redis context. The client receives only a hardcoded confirmation string. 16

LIST OF FIGURES

List of Tables

3.1	Antares AI runtime services	15
-----	---------------------------------------	----

